Application of Saddlepoint Approximation in Honing Forecast

Kenivia

Feb 2026

Contents

1 Introduction **2**

2 Setup **2**
2.1 Basics 2
2.2 Special honing 3
2.3 What we're computing 3

3 Special honing: p_k **3**

4 Contribution from one cost dimension: $\mathbb{E}[G_i(X'_k)]$ **4**

5 Probability that we exceed budget: $\mathbb{P}(Y < b)$ **5**
5.1 Cumulant generating function $K(s)$ 5
5.2 Find s such that $K'(s) \approx b$ 6
5.3 $\mathbb{P}(Y < b) =$ magic formula 6
5.4 What about Y^*? 6

1 Introduction

This document will go more into more detail about what exactly we are computing, then go through how we do it. This is aimed at an audience who fall in the intersection of Lost Ark players and aspiring mathematicians. As such, we will use rigorous notations to make our statements and logic clear without losing track of our greedy goblin goal of saving gold.

2 Setup

We must first endure some boring definitions, after which we will find that the math is rather simple up until the very last part.

2.1 Basics

Let $U_j = U_1, U_2, \ldots, U_N$ be the upgrades we want to perform, which we will view as discrete random variables.Note that the upgrades are independent, and take value in $0, 1, \ldots n_j$, representing the number of attempts needed for each upgrade.

Let $X = (U_1, U_2, \ldots, U_N)$ be the random vector representing the overall outcome[1].

Let $C_i(x)$ be the material cost function for material i, where $i \in 1, 2, \ldots, M$. For ordinary mats, this is of the form $C_i(x) = u_i + c_i \cdot x$ for some constants u_i and c_i. However, this is arbitrary for juices.

Let b_i be the "budget", or the amount of material i that we own.

Let m_i and l_i be the market price and leftover value of material i.

Let $g_i(x)$ be our gold cost function, defined as follows[2]:

$$g_i(x) = \begin{cases} (x - b_i) \cdot m_i & \text{if } x > b_i \\ (x - b_i) \cdot l_i & \text{otherwise} \end{cases}$$

As in, every unit of material i that we consume beyond our budget incurrs m_i gold cost, and every unit of i that we have leftover is worth l_i gold. By default, l is 0 (our budget is untradable).

[1] We simplify the multi-step process into a single realization of the RV. While our end goal is to find the "optimal move", this simplification answers a different question: "What is the optimal plan if you couldn't change your plan before going into attempting upgrades?" The good news is that the first part of this plan happens to be the optimal move that we take, and whatever steps we follow is the optimal given the information we had. As such, there may be a better move if we assume that our user will keep updating their progress! However I think it's safe to assume that few are bothered to do so, and this simplification also makes our evaluation signficantly easier (I wouldn't even know where to start otherwise).

[2] The signs are the other way round in the code, this is consistent with the UI.

As such, we can consider the gold cost due to material i:

$$G_i(X) = g_i(\sum_{j=1}^{N} C_i(U_j))$$

Note that g_i is applied to the sum, not the cost of the individual upgrades.

2.2 Special honing

There's an additional layer of complexity we must define:

Let U'_k be the "skipped" version of the random variable. This is just a deterministic constant equal to the number of normal honing attempts the user did before doing special hones, usually equal to 0 (but does not necessarily mean $C_i(U'_k) = 0$).

Let $X'_k = (U'_1, \cdots, U'_k, U_{k+1}, \cdots, U_N)$, the overall outcome with k skipped upgrades.

Let p_k be the chance that we skip exactly k upgrades, where $k \in 0$, 1, $\ldots, N$

2.3 What we're computing

To describe this in English (neglecting the grammar issues with negative costs), this would be "Average gold cost incurred due to needed mat". Making use of the notations we've built up, this is

$$\sum_{k=0}^{N}(p_k \cdot \sum_{i=1}^{M} \mathbb{E}[G_i(X'_k)]).$$

The word "Average" is doing a lot of heavy lifting here. Note that since expectation is distributive over sums[3], we are allowed to add up the contributions of each material dimension (index i in the sum above).

There are 3 layers of multi-dimension-ness at work here: Special honing, different material types, and different upgrades. The first two are trivial and have been written as sums. However, G_i is a non-linear function that depends on the overall outcome, not just the outcome of a single upgrade, so we cannot factor it out. And so our journey of unwrapping this bundle of joy begins.

3 Special honing: p_k

We must recognize that we have made a simplification here - we assume that we must keep doing special hones on one upgrade until either we run out or

[3]This holds even for non-independent random variables, which the cost dimensions, $G_i(X'_k)$, are not (because they depend on the same underlying RV).

we succeed. (The alternative being attemping A for a times, then B for b times, then back to A etc etc)

This assumption has a few motives / implications:

1. The decision space of special hones is just the permutations of $U_1, U_2, \ldots, U_N$. This makes the optimizer's job a lot easier, even if this might forbid some better configurations.

2. Succeeding k implies that we succeeeded $k-1$, which has 2 effects:

   (a) This drastically reduces the number of possibilities we need to consider, and therefore the number of times we need to evaluate the inner sum. As in, we only need to consider $(fail, f, f)$, $(success, f, f)$, (s, s, f), (s, s, s) etc, as opposed to the cartesian product $\{s, f\}^n$. This reduces the complexity from 2^n to $(n+1)$. This is by far the biggest concern[4].

   (b) This makes the computation of p_k a lot easier. This is done in special.rs using some dynamic programming.

3. This is easier for the user to follow.

4 Contribution from one cost dimension: $\mathbb{E}[G_i(X'_k)]$

Unfortunately, for a non-linear f, $\mathbb{E}[f(X)] \neq f(\mathbb{E}[X])$, which is the whole reason why we're here. Recall that:

$$G_i(X) = g_i(\sum_{j=1}^{N} C_i(U_j))$$

where g_i is non-linear. If this was a general function, this would require either a (brute force) convolution of random variable or something like Fast Fourier Transform(I think). However, we know that our g_i is piecewise linear:

$$g_i(x) = \begin{cases} (x - b_i) \cdot m_i & \text{if } x > b_i \\ (x - b_i) \cdot l_i & \text{otherwise} \end{cases}$$

and so we can do something more clever. For convenience define

$$Y_i = \sum_{j=1}^{N} C_i(U_j), \text{so } G_i(X) = g_i(Y_i).$$

[4] Although if we limit our decision space to allow for a limited number of switches, this COULD be viable. However even allowing for 1 switch doubles the amount of SA evaluations, so this would only be vaguely viable for a small number of upgrades.

We can notice that :

$$\mathbb{E}[g_i(Y_i)] = m_i \cdot \mathbb{E}[(Y_i - b_i) \cdot I(Y_i > b_i)] + l_i \cdot \mathbb{E}[(Y_i - b_i) \cdot I(Y_i < b_i)],$$

where I is the indicator function. From here, we will drop the subscripts $_i$ as we're focusing on one material type dimension. We can split this into three parts:

$$\begin{aligned}\mathbb{E}[g(Y)] &= m \cdot \mathbb{E}[Y - b] + (m - l) \cdot E[(Y - b) \cdot I(Y < B)] \\ &= m \cdot \underbrace{\mathbb{E}[Y - b]}_{\text{part a}} + (m - l) \cdot (\underbrace{E[Y \cdot I(Y < B)]}_{\text{part b}} - \underbrace{E[b \cdot I(Y < B)]}_{\text{part c}})\end{aligned}$$

Where part a) is thankfully trivial to calculate because Y is a simple sum of RVs, making $\mathbb{E}$ distributive.

For part b), we define the biased distribution Y^*, which has the probabily distribution $p^* = p \cdot y$ for each y in the support of Y. In other words:

$$\mathbb{P}(Y^* = y) = \frac{y\mathbb{P}(Y = y)}{\mathbb{E}[Y]},$$

such that Y^* satisfies:

$$\mathbb{E}[Y \cdot I(Y < b)] = \mathbb{P}(Y^* < b) \cdot \mathbb{E}[Y].$$

,

Part c) is a simpler version of part b), we have:

$$\mathbb{E}[b \cdot I(Y < b)] = \mathbb{P}(Y < b) \cdot b.$$

As such, all we need is some way to compute $\mathbb{P}(Y^* < b)$ and $\mathbb{P}(Y < b)$!

5 Probability that we exceed budget: $\mathbb{P}(Y < b)$

We use a procedure known as saddleppoint approximation to perform this computation. I will describe the steps below:

5.1 Cumulant generating function $K(s)$

We compute $K(s)$, where K is different for a particular Y but we will spare the subscripts. $K(s)$ is defined as follows:

$$K(s) = \log(\mathbb{E}[e^{sY}]).$$

K has the property that it is distributive over addition. Recall that $Y_i = \sum_{j=1}^{N} C(U_j)$, so we have:

$$K(s) = \sum_{j=1}^{N} \log(\mathbb{E}[e^{sC(U_j)}]).$$

This property allows us to compute the individual summands and add them up at the end to obtain $K(s)$. Furthermore, it is also easy to compute its derivatives $K'(s)$, $K''(s)$, $K'''(s)$, $K^{(4)}(s)$ (we stop at the 4th cumulant for floating point reasons).

5.2 Find s such that $K'(s) \approx b$

Since we have access to higher derivatives of $K'(s)$, we can use a generalized version of newton's algorithm for root finding, known as Householder's method. My particular implementation uses a bunch of heuristics to avoid non-convergence that are complete trial and error, so there may be faster implementations.

5.3 $\mathbb{P}(Y < b) =$ magic formula

The specific version of Saddlepoint approximation implemented is as follows:

$$\mathbb{P}(Y < b) = \begin{cases} \Phi(w) + \phi(w)(\frac{1}{w} - \frac{1}{u}) & \text{if } x \neq \mu \\ \Phi(z) - \phi(z)\left[\frac{\kappa_3}{6}H_2(z) + \frac{\kappa_4}{24}H_3(z) + \frac{\kappa_3^2}{72}H_5(z)\right] & \text{if } x \approx \mu, \end{cases}$$

$$\begin{aligned} \text{where } w &= \operatorname{sgn}(s)\sqrt{2(sb^+ - K(s))} \\ u &= 2\delta^{-1}\sinh(s\delta/2)\sqrt{K''(s)} \\ z &= (b^+ - \mu)/\sigma \\ \kappa_n &= K^{(n)}(0)/\sigma^n \\ \delta &= \text{lattice span of the support of Y} \\ b^+ &= b + \delta/2 \\ H_n &= \text{Hermite polynomial of degree } n. \end{aligned}$$

The derivation of this formula can be found in Saddlepoint Spproximations with Applications [1], specifically the base form is stated in Section 1.2.1, the continuity corrected version in Section 2.4.4, and the limiting case in Section 5.2.3.

5.4 What about Y^*?

Since Y^* is not a sum of random variables, K is not distributive over its summands, so naively computing the cumulant generating function of Y^* would require a direct convolution. Fortunately, It turns out the cumulant of Y^* and Y can be related by the following:

$$K_Y^*(s) = K_Y(s) + \log(K_Y'(s)) - \log(\mu).$$

Higher cumulants can be found similarly via closed form formulae.

References

[1] Ronald W Butler. *Saddlepoint approximations with applications*, volume 22. Cambridge University Press, 2007.